Data-Structures & Algorithms

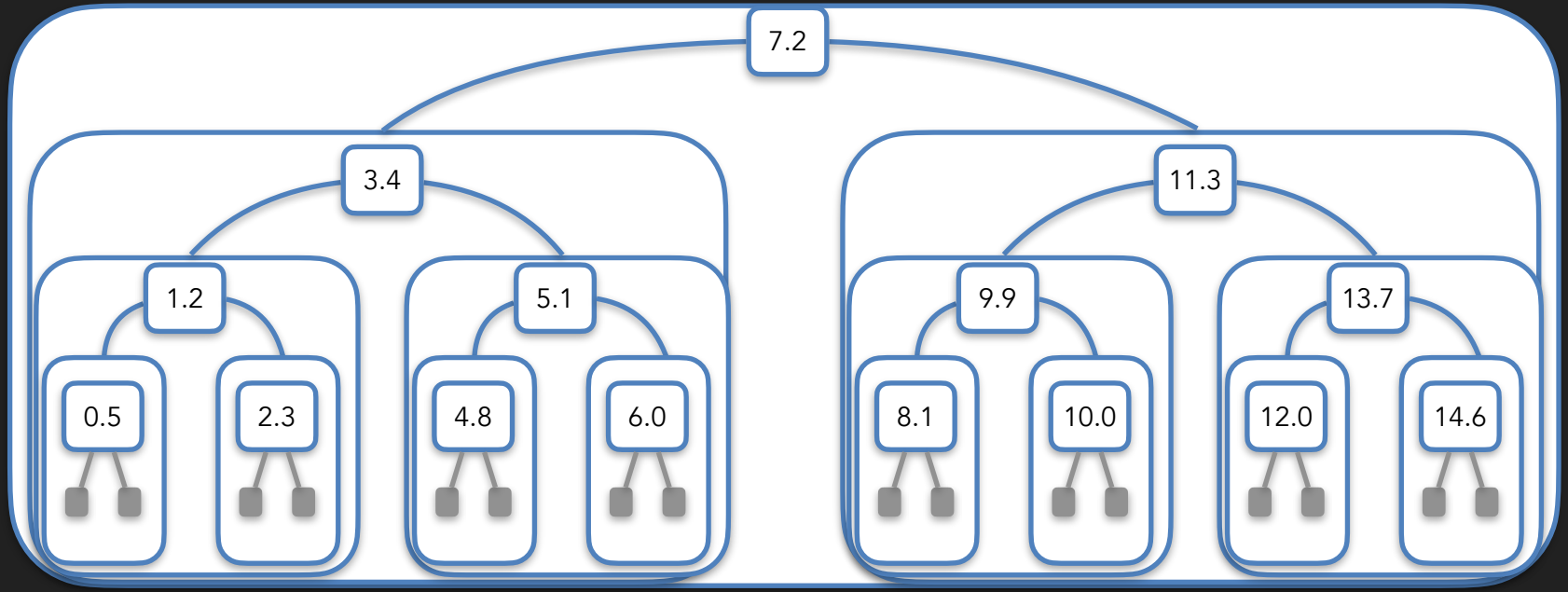
with

Manoj Prabhakaran

Lecture 12

Binary Search Trees

Binary Search Trees

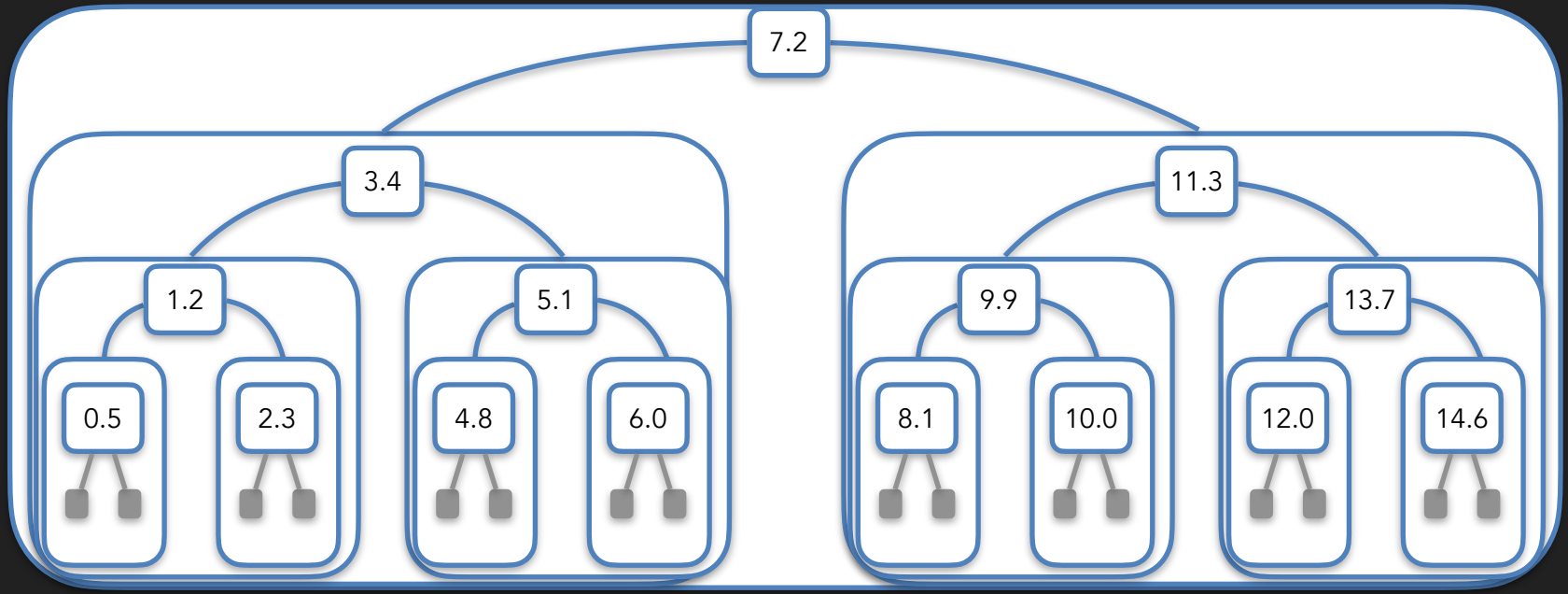


Defining a BST: A binary tree such that for any node with key K,

all nodes in its left subtree have keys < K, and

all nodes in its right subtree have keys ≥ K.

Binary Search Trees



in-order: 0.5 1.2 2.3 3.4 4.8 5.1 6.0 7.2 8.1 9.9 10.0 11.3 12.0 13.7 14.6

In a BST, the in-order sequence is sorted

Inserting into a BST

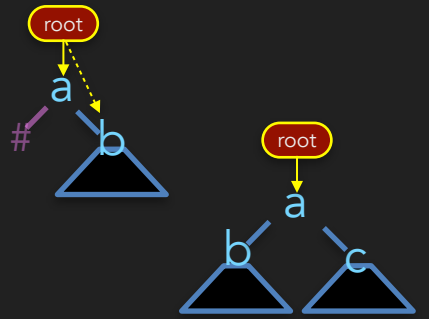
- When searching for a key not present in the tree, the search terminates at a leaf where the key could have been
- To insert a key:
  - First search for the key
    - Ignore if found! Pretend the key we are searching for is slightly larger (because equal keys go to the right sub-tree)
  - Grab the parent of the leaf where the search terminates
  - Create a new node with the key and insert it under this parent

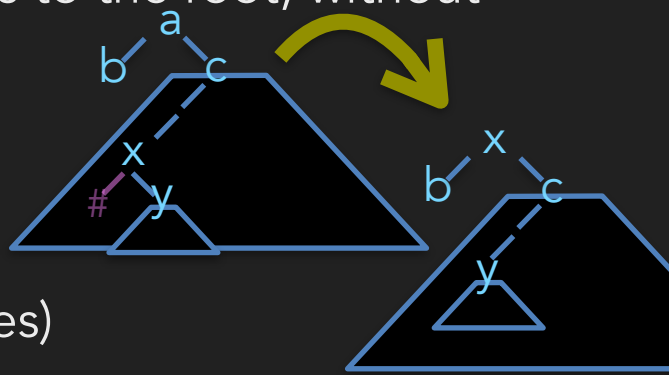
Inserting into a BST

```
struct node{int key; node *parent, *left=nullptr, *right=nullptr};
struct tree {
  node* root = nullptr;
  void insert(int v) {
   // search for the parent whose child the new node will be
    node* parent = nullptr;
    for(node* curr = root; curr!=nullptr; ) {
      parent = curr; // update parent before updating current
      if(v < curr->key) curr = curr->left;
      else /*if(v>=curr->key)*/ curr = curr->right;
    }
    // create and attach the new node
    node* newnode = new node{v,parent};
    if(parent == nullptr) root = newnode; // was an empty tree
    else if (v < parent->val) parent->left = newnode;
    else parent->right = newnode;
  }
};
```

Deleting from a BST

- Consider deleting the root from a (non-empty) tree
  - Has only leaves as its children? Simply make it a leaf (empty the tree)
  - Has one leaf and one internal node as children?
    - Promote the child internal node to be the root
  - If both children are internal nodes?
    - Plan: Bring a node from one of the sub-trees to the root, without moving any other node!
    - Minimum node on the right sub-tree
      - Detach that node from the sub-tree that it is root of (using one of the first two rules)





Deleting from a BST

```
struct node{int key; node *parent, *left=nullptr, *right=nullptr};
struct tree {
  node* root = nullptr;
  node*& where(node* p) { // which pointer in the tree is p
    if(p==root) return root;
    return (p->parent->left==p)? p->parent->left : p->parent->right;
  }
  node* minimum(node* p) { // min in subtree under p (p!=nullptr)
    while(p->left) {p = p->left;} // go as left as possible
    return p;
  }
  void detach(node* p); // detach but don't delete p (p!=nullptr)
  void erase(node* p) {if(!p) return; detach(p); delete p;}
};
```

Deleting from a BST

```
void tree::detach(node* p) { // detach but don't delete p (p!=nullptr)
  node*& pp = where(p);      // pp = where p is recorded in the tree
  if(!p->left || !p->right){ // if at least one leaf child
    pp = p->left ? p->left : p->right; // promote the other child
    if(pp) pp->parent = p->parent;
    return;
  }
  node* q = minimum(p->right);       // if no leaf child, find successor q
  detach(q);    // since q->left==nullptr, this will use the first case
  // replace p with q. update pointers to p
  pp = q; p->left->parent = q;       // p->left!=nullptr
  if(p->right) p->right->parent = q; // detach(q) can set p->right=nullptr
  // update pointers from q
  q->left = p->left; q->right = p->right; q->parent = p->parent;
  p->left = p->right = p->parent = nullptr; // optional: clean up p
}
```

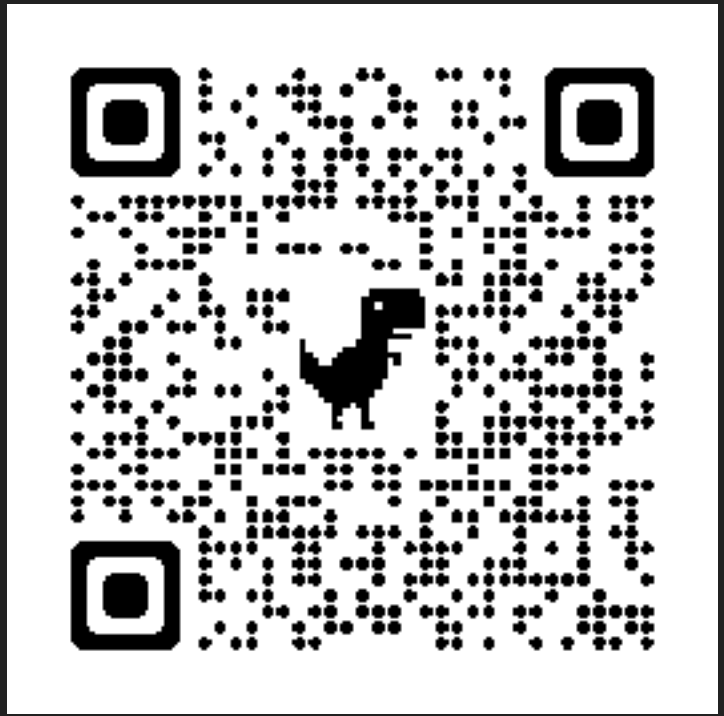
Attendance Break!

Login using CSE LDAP id at
https://www.cse.iitb.ac.in/course_vm/cs433

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front



If you are not in the classroom, a report will be sent to DDAC!